

Hadoop与Spark

Hadoop入门



Hadoop入门

◆ Hadoop简介

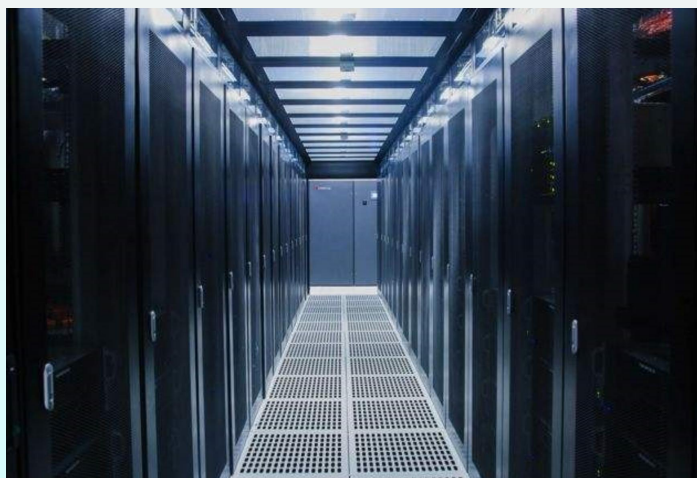
- ◆ Apache Hadoop是存储和处理大数据的开源软件框架
- ◆ Hadoop项目，最初由Doug Cutting和Mike Cafarella于2005年创建，其最初的目标，是提供Nutch搜索引擎的分布式处理能力
- ◆ 目前Doug Cutting是Cloudera的首席架构师，Cloudera是一家基于Hadoop开源软件，提供增值开发和服务的创业公司



Hadoop入门

◆ Hadoop简介

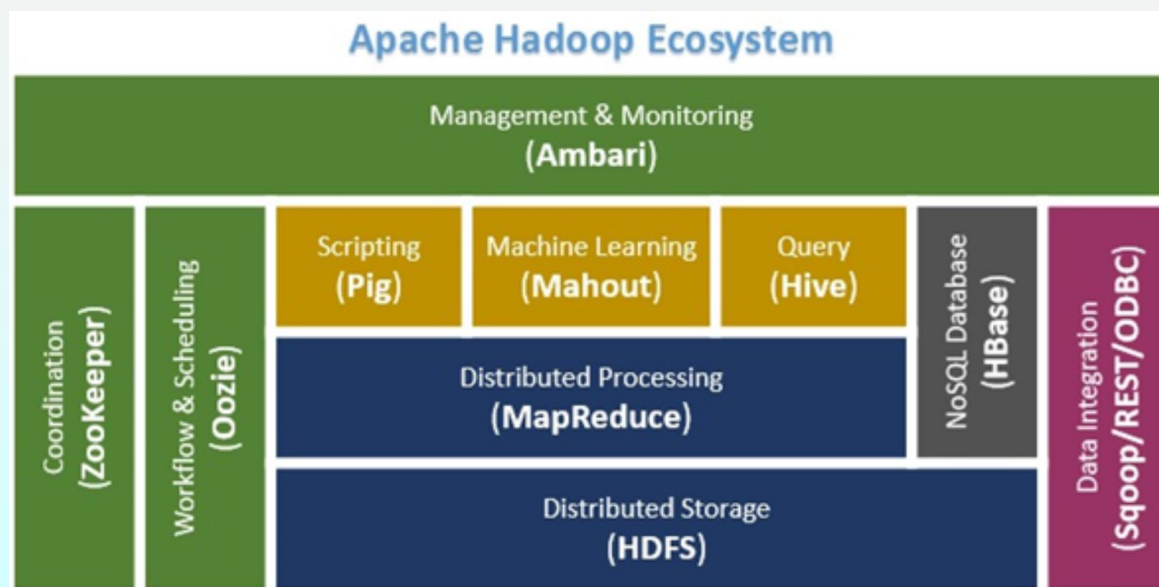
- ◆ 在扩展性(Scalability)方面, Hadoop能够在**上千台机器**组成的集群上运行
 - ◆ 大规模集群的可靠性, 不能仅仅靠硬件来保证, 因为节点的失败、网络的失败等状况不可避免
 - ◆ 为了能够在大规模集群上顺利运行, Hadoop的所有模块, 其设计原则基于这样的基本假设, 即硬件的失败在所难免, 每个节点都没有那么可靠, 可能发生节点失败状况, 软件框架应该能够自动检测和处理这些失败情况
 - ◆ Hadoop通过软件, 在大规模集群上提供高度的可用性(High Availability)



Hadoop入门

◆ Hadoop的模块

- ◆ Hadoop软件框架，包含如下主要模块：
(1) **Hadoop Common**，这个模块包含了其它模块需要的库函数和实用函数。
(2) **Hadoop Distributed File System (HDFS)**，这是在由普通服务器组成的集群上运行的分布式文件系统，支持大数据的存储。通过多个节点的并行I/O，提供极高的吞吐能力。
(3) **Hadoop MapReduce**，是一种支持大数据处理的编程模型。
(4) **Hadoop YARN**，这是Hadoop2.0的基础模块，它本质上是一个资源管理和任务调度软件框架
 - ◆ 它把集群的计算资源管理起来，为调度和执行用户程序提供支持

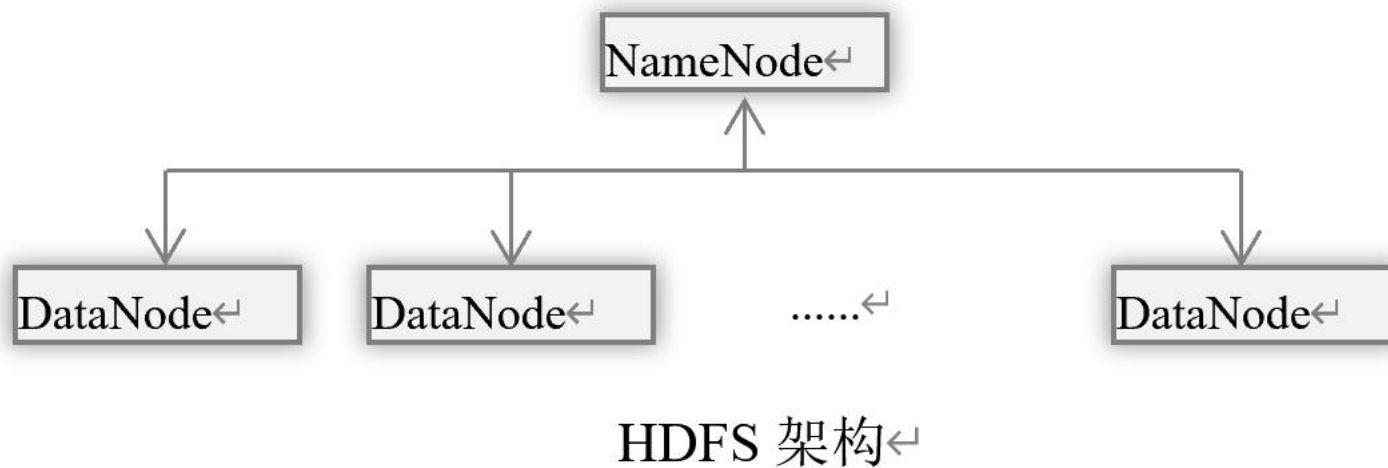


pause



Hadoop入门

- ◆ Hadoop分布式文件系统
 - ◆ 架构
 - ◆ Hadoop分布式文件系统(Hadoop Distributed File System, HDFS), 是一个分布式的、高度可扩展的文件系统
 - ◆ 一个HDFS集群, 一般由一个NameNode和若干DataNode组成, 分别负责元信息的管理和数据块的管理



Hadoop入门

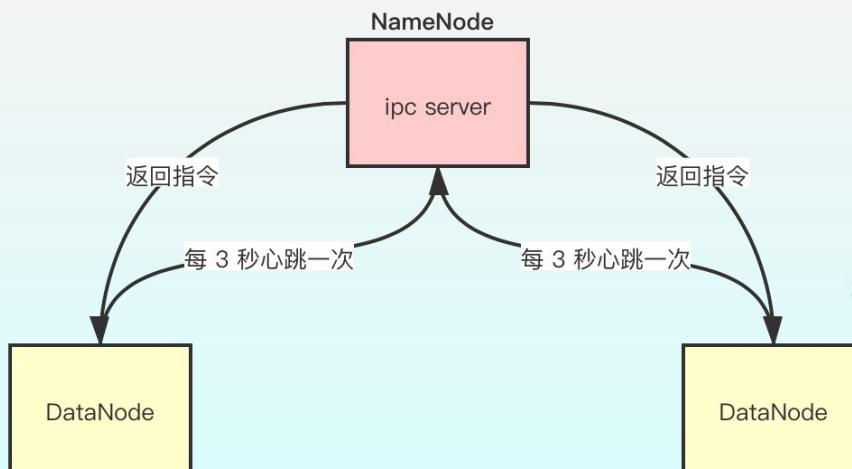
- ◆ Hadoop分布式文件系统
 - ◆ NameNode和DataNode
 - ◆ **NameNode**是HDFS系统中的管理者
 - ◆ (1) 它负责管理文件系统的命名空间：维护文件系统的文件树以及全部的文件和文件夹的元数据
 - ◆ (2) 这些信息存储在NameNode维护的两个本地磁盘文件里：命名空间镜像文件(**FsImage**)和编辑日志文件(**EditLogs**)
 - ◆ (3) NameNode中还保存了每一个文件及其数据块所在的DataNode的相应关系，这些信息被用于其它功能组件查找文件(数据块)所在的DataNode
 - ◆ (4) 接收客户端的请求：上传、下载、创建目录等
 - ◆ **DataNode**是HDFS中保存数据的节点
 - ◆ (1) 存储数据块
 - ◆ (2) 负责客户端对数据块的IO请求
 - ◆ (3) DataNode之间相互通信，执行数据块复制
 - ◆ (4) DataNode定时和NameNode进行心跳通信，接受NameNode的指令

Hadoop入门

- ◆ Hadoop分布式文件系统
 - ◆ 容错
 - ◆ HDFS支持TB级甚至PB级大小文件的存储
 - ◆ 它把文件划分成数据块(Block)，分布到多台机器上进行存储
 - ◆ 为了保证系统的可靠性，HDFS把数据块在多个节点上进行复制(Replicate)
 - ◆ 如果HDFS采用的复制因子(Replicate Factor)为3，那么每个数据块有三个副本，被保存到三个节点上，其中的两个节点在同一个机架内，另外一个节点一般在其它机架上
 - ◆ DataNode之间可以拷贝数据副本，从而重新平衡每个节点存储的数据量，并且保证系统的可靠性(保证每个数据块都有足够的副本)

Hadoop入门

- ◆ Hadoop分布式文件系统：心跳机制
 - ◆ DataNode默认每3秒向NameNode汇报一次，向NameNode汇报状态信息，并且返回NameNode对DataNode的操作命令
 - ◆ 如果某一时刻DataNode不汇报，可能宕机
 - ◆ NameNode超过十分钟没有收到DataNode的心跳，则认为该节点不可用



Hadoop入门

◆ HDFS优缺点

◆ 优点

- ◆ (1) 高容错性
- ◆ (2) 适合批处理
- ◆ (3) 适合大数据处理
- ◆ (4) 一次写入，多次读取，不能修改，只能追加
- ◆ (5) 可构建在廉价机器上

◆ 缺点

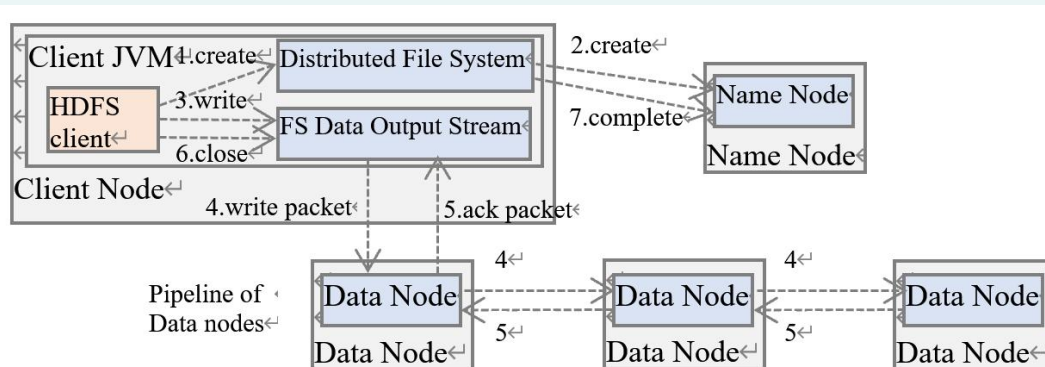
- ◆ (1) 不适合低延时数据访问
- ◆ (2) 无法高效地对大量小文件进行存储
- ◆ (3) 一个文件只能有一个写，不允许多个线程同时写
- ◆ (4) 仅支持数据 append（追加），不支持文件的随机修改

Hadoop入门

◆ Hadoop分布式文件系统

◆ 写文件

- ◆ (1) 客户端程序调用DistributedFileSystem的create()方法
- ◆ (2) DistributedFileSystem向NameNode发起一个远程过程调用(Remote Procedure Call, RPC)，由其在文件系统的命名空间里，创建一个新文件。这时，该文件还没有任何数据块
 - ◆ NameNode进行一系列的检查，保证文件并未存在(才可以新建)，而且客户端有权限创建该文件。如果检查通过，那么NameNode为新文件创建一个新的记录，保存其信息，否则文件的创建失败
 - ◆ DistributedFileSystem返回FSDataOutputStream 给客户端程序，以便其开始写入数据。FSDataOutputStream 包含一个DFSOutputStream 对象，它负责和NameNode以及DataNode的通讯



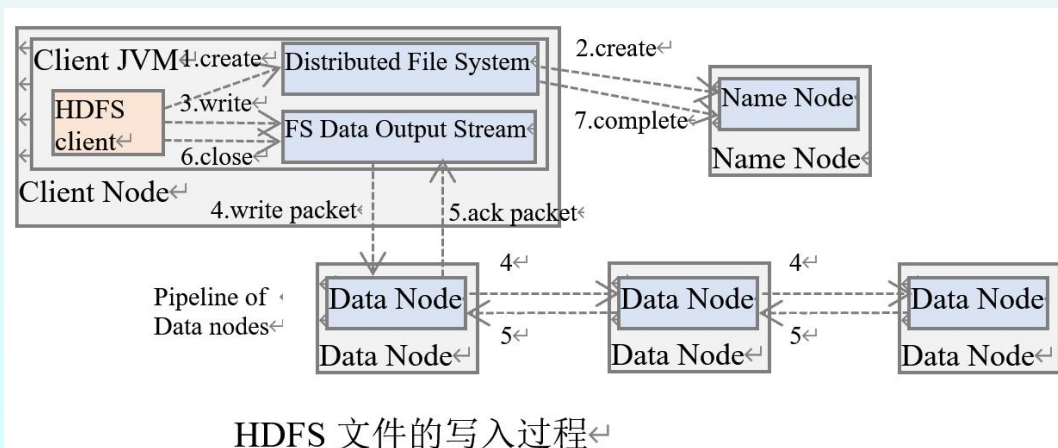
HDFS 文件的写入过程

Hadoop入门

◆ Hadoop分布式文件系统

◆ 写文件

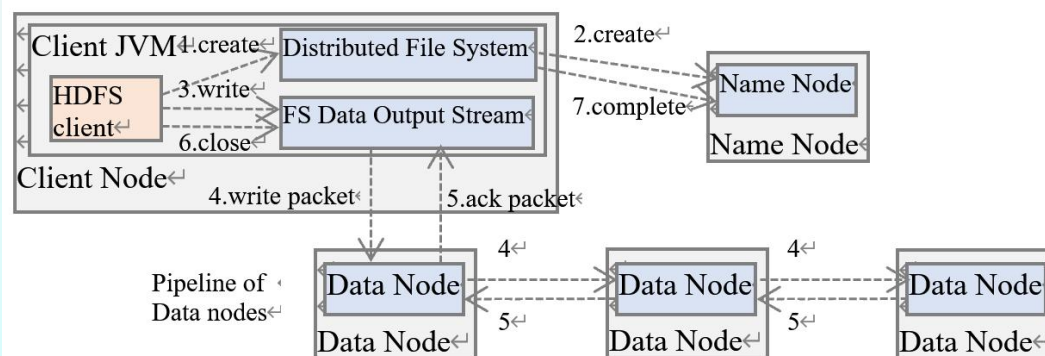
- ◆ (3) 当客户端开始写入数据，DFSOutputStream把数据分解成数据包(Packet)，并且写入一个内部队列，称为数据队列(Data Queue)。DataStreamer消费这个数据队列，请求NameNode为新的数据块分配空间，即选择一系列合适的DataNode，用于存放各个数据块的各个副本。
 - ◆ 存放各个副本的DataNode形成一个流水线(Pipeline)，我们假设复制因子(Replica Factor)是3，于是在流水线上有三个节点
 - ◆ DataStreamer把数据包发送到第一个DataNode，这个DataNode保存数据包，并且转发给流水线上的第二个DataNode...
 - ◆ 当写入数据已经超过一个数据块的大小时，DataStreamer向NameNode申请为新的数据块分配空间



Hadoop入门

◆ Hadoop分布式文件系统

- ◆ 写文件
- ◆ (4) 第二个DataNode保存这个数据包，并且转发给第三个(最后一个)DataNode
- ◆ (5) DFSOutputStream同时维护一个数据包的内部队列(Internal Queue)，用于等待接收DataNode的应答信息，称为Ack Queue
 - ◆ 当某个数据包已经被流水线上的所有DataNode应答以后，它才被删除(从Ack Queue上)。
- ◆ (6) 当客户端程序完成数据写入，它调用数据流的close()方法。
- ◆ (7) 客户端把所有剩余的数据包发送到DataNode流水线上，并且等待应答信息，最后联系NameNode，告诉它文件结束



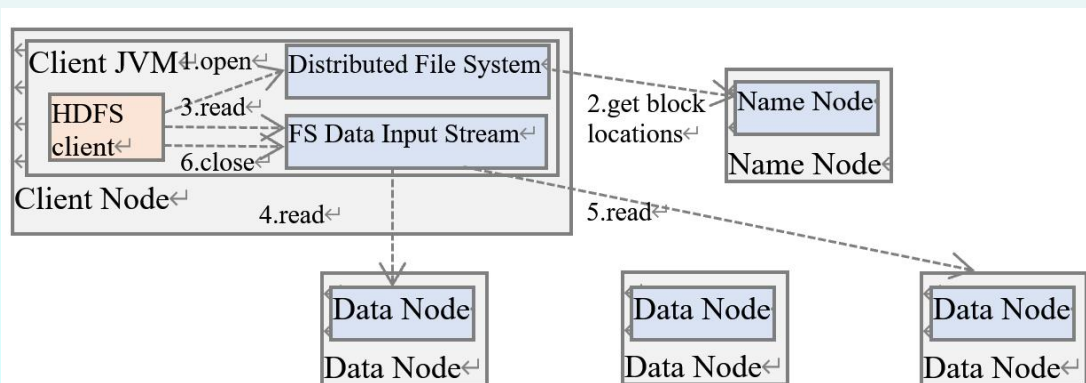
HDFS 文件的写入过程

Hadoop入门

◆ Hadoop分布式文件系统

◆ 读文件

- ◆ (1) 客户端程序通过调用FileSystem对象的open()方法，打开文件，获得DistributedFileSystem类的一个实例。
- ◆ (2) DistributedFileSystem 通过RPC调用NameNode，获得文件首批数据块的位置信息(Locations of Blocks)
 - ◆ 对于每个数据块来讲，NameNode会返回拥有这个数据块的副本的所有DataNode的地址
 - ◆ DistributedFileSystem返回FSDataInputStream类的一个实例，这是一个输入流(Input Stream)，支持文件的定位和数据的读取，使得客户端可以读取文件数据
 - ◆ FSDataInputStream 包含了一个DFSInputStream对象，负责管理客户端对HDFS的DataNode和NameNode的存取



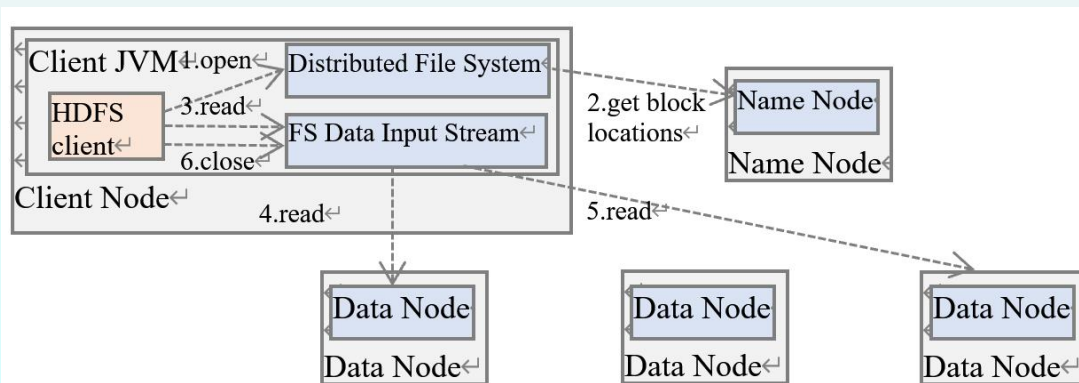
HDFS 文件的读取过程

Hadoop入门

◆ Hadoop分布式文件系统

◆ 读文件

- ◆ (3) 客户端程序从输入流上调用函数read()。由于DFSInputStream已经保存了文件首批若干数据块所在的DataNode的地址，DFSInputStream连接到最近的(Closest)DataNode，读取文件的第一个数据块
- ◆ (4) 数据从DataNode源源不断传送回客户端程序，而客户端程序则不断地调用数据流的read()方法
- ◆ (5) 当到达数据块的末尾的时候，DFSInputStream将关闭到DataNode的连接，然后寻找下一个数据块的最优的DataNode，以便进行后续数据块的读取
 - ◆ DataNode的选择对客户端程序来讲是透明的，客户端程序只是从一个连续的数据流进行读取



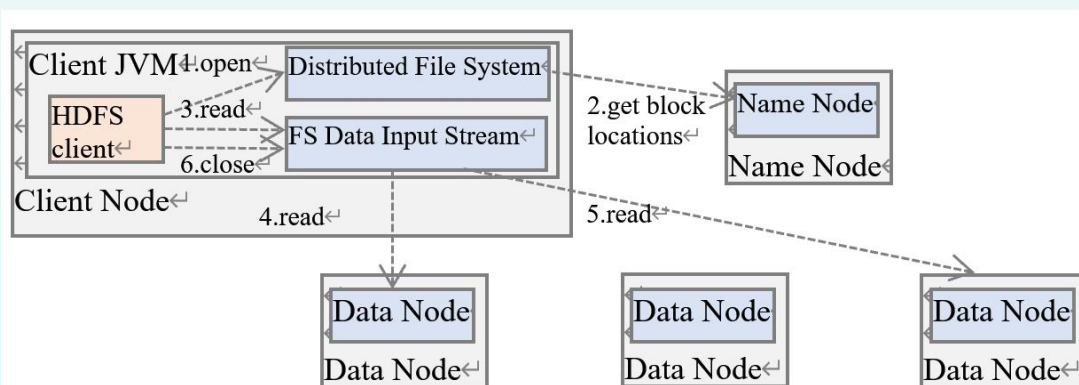
HDFS 文件的读取过程

Hadoop入门

- ◆ Hadoop分布式文件系统

- ◆ 读文件

- ◆ 客户端按照顺序读取各个数据块。当客户端不断读取数据流的时候，在数据块的边界，DFSInputStream不断创建到保存后续数据块的DataNode的连接
 - ◆ DFSInputStream同时向NameNode询问和提取下一批数据块的DataNode的位置信息
 - ◆ (6) 当客户端完成文件的读取，它调用FSDataInputStream实例的close()方法



HDFS 文件的读取过程

pause



Hadoop

◆ Name Node(NN)

- ◆ FS image: 保存了最新的元数据检查点
- ◆ Edits: 主要是在NameNode已经启动情况下对HDFS进行的各种更新操作进行记录
- ◆ $\text{FS image(old)} + \text{Edits} = \text{FS image(new)}$

Hadoop

◆ Secondary Name Node(SN)

- ◆ **Secondary Name Node**的作用就是帮助**NameNode**进行**Edits**和**FsImage**的合并工作
- ◆ 由于Edits中记录的操作会越来越多，Edits文件会越来越大，导致NameNode在启动加载Edits时会很慢，所以需要**对Edits和Fsimage进行合并**（所谓合并，就是将Edits和Fsimage加载到内存中，照着Edits中的操作一步步执行，最终形成新的Fsimage）
- ◆ **Secondary Name Node**的作用就是帮助**NameNode**进行**Edits**和**Fsimage**的合并工作

Hadoop

- ◆ **Secondary Name Node(SN)**
 - ◆ **Secondary Name Node**的作用就是帮助**NameNode**进行**Edits**和**FsImage**的合并工作
 - ◆ (1) SecondaryNameNode首先会询问NameNode是否需要CheckPoint（触发CheckPoint需要满足两个条件中的任意一个，定时时间到和Edits中数据写满了）
 - ◆ (2) SecondaryNameNode执行CheckPoint操作
 - ◆ (3) 首先会让NameNode给正在写的Edits打标记，并生成一个新的空的edits_inprogress文件，以后所有新的操作都写入edits_inprogress
 - ◆ (4) 其他未合并的Edits和Fsimage会拷贝到SecondaryNameNode的本地

Hadoop

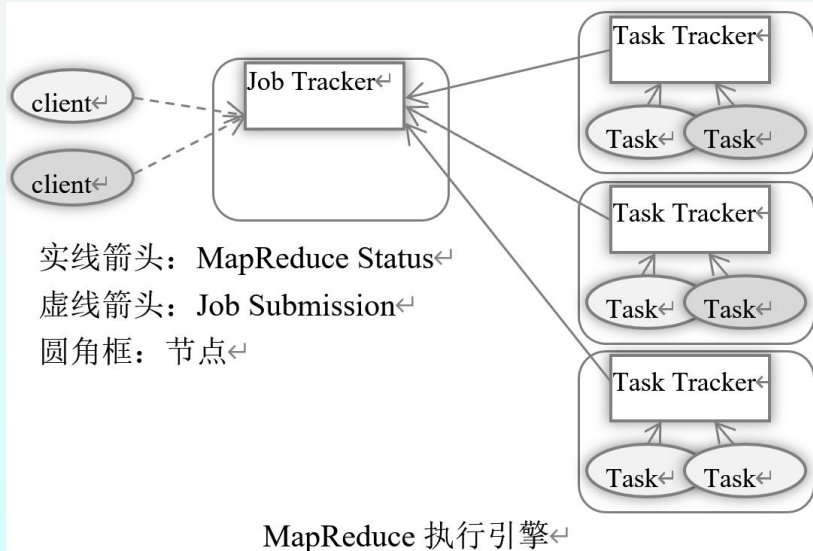
- ◆ Secondary Name Node(SN)
 - ◆ **Secondary Name Node**的作用就是帮助**NameNode**进行**Edits**和**FsImage**的合并工作
 - ◆ (5) 在SecondaryNameNode 上将拷贝的Edits和Fsimage加载到内存中进行合并
 - ◆ (6) 生成新的fsimage.chkpoint文件，包含上一步合并的数据信息
 - ◆ 将fsimage.chkpoint拷贝给NameNode
 - ◆ (7) 在NameNode上将fsimage.chkpoint重命名为Fsimage后替换掉原来的Fsimage
 - ◆ (8) NameNode在启动时就只需要加载之前未合并的Edits和Fsimage即可，因为合并过的Edits中的元数据信息已经被记录在Fsimage中

pause



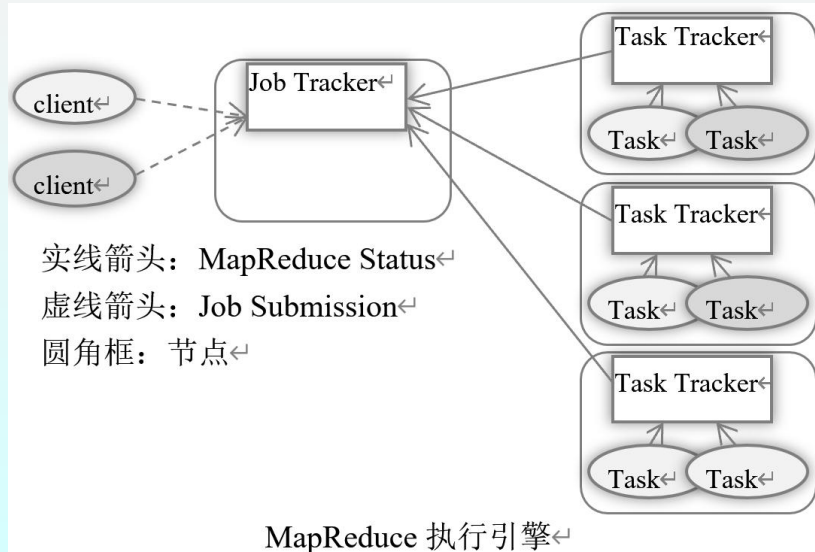
Hadoop入门

- ◆ MapReduce执行引擎
 - ◆ **MapReduce执行引擎**运行在分布式文件系统HDFS之上，它包括JobTracker和TaskTracker两个主要的组成部分，分别运行在NameNode和DataNode上
 - ◆ 用户提交的数据处理请求，**称为一个作业(Job)**，由JobTracker分解为数据处理任务(Task)，分发给集群里的相关节点上的TaskTracker运行。
 - ◆ 客户端程序把作业提交给JobTracker以后，JobTracker把数据处理任务，发送给整个集群各个节点的TaskTracker



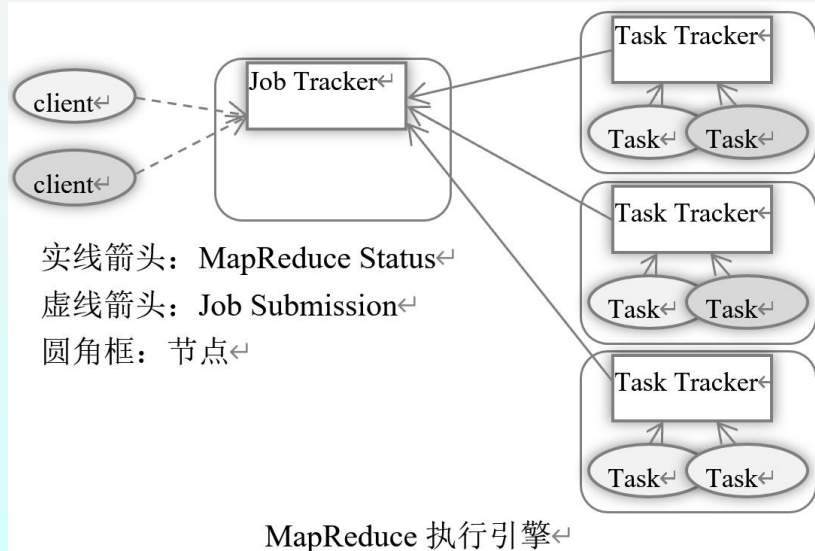
Hadoop入门

- ◆ MapReduce执行引擎
 - ◆ 在HDFS里, JobTracker通过HDFS NameNode, 知道哪些节点包含将要处理的各个数据块, 也就是它了解数据块的存放位置(Location)
 - ◆ 如果任务(Task)不能发送到数据块所在的节点, 比如因为该节点目前Task Slot(任务槽, 即每个TaskTracker可以运行的task数量)已经用完, 那么系统优先把任务推送到同一机架里的其它节点, 该节点保留了数据块的另外一个副本(Replica)



Hadoop入门

- ◆ MapReduce执行引擎
 - ◆ 如果TaskTracker失败或者运行超时，它负责的任务被JobTracker重新调度到其它的TaskTracker上
 - ◆ 在TaskTracker运行过程中，它向JobTracker每隔几分钟发送一个心跳信号(Heart Beat)，以便报告它的存活状态



Hadoop入门

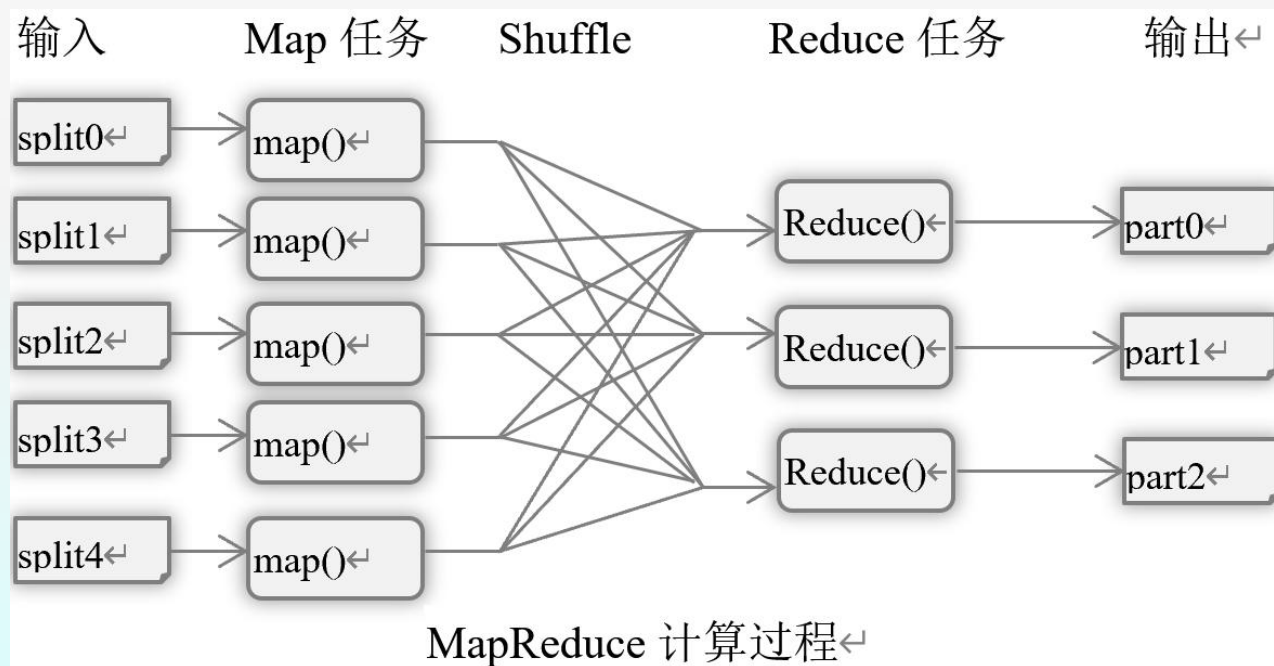
- ◆ MapReduce执行引擎
 - ◆ 被分配了任务的TaskTracker从HDFS文件系统中取出所需的文件，包括JAR程序文件和任务相应的数据文件，存入本地磁盘，并启动一个TaskRunner程序实例准备执行任务。
 - ◆ TaskRunner在一个新的Java虚拟机中依据任务类型创建出MapTask或ReduceTask进行运算
 - ◆ 在新的Java虚拟机中执行MapTask和ReduceTask的原因，是避免这些任务的执行异常影响TaskTracker的正常执行。MapTask和ReduceTask会定时与TaskRunner进行通信报告进度，直到任务完毕。
 - ◆ 每一个TaskTracker节点可执行Map任务和Reduce任务的数量也是有限的，即每一个TaskTracker有两个固定数量的任务槽，分别响应Map任务和Reduce任务
 - ◆ 在进行任务分配时，JobTracker优先填满TaskTracker的Map任务槽，即只要有空暇Map任务槽，就分配一个Map任务，Map任务槽满了之后，才分配Reduce任务
 - ◆ 注意，一个MapReduce作业的Map Tasks和Reduce Tasks有先后依赖关系

pause



Hadoop入门

- ◆ MapReduce计算模型
 - ◆ MapReduce并行编程模型把计算过程分解为两个主要阶段，即Map阶段和Reduce 阶段



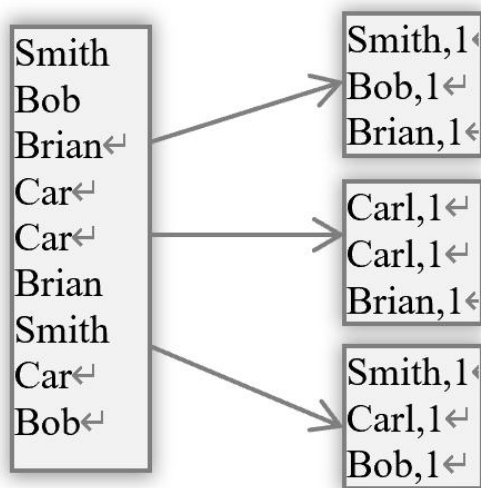
Hadoop入门

◆ MapReduce计算模型

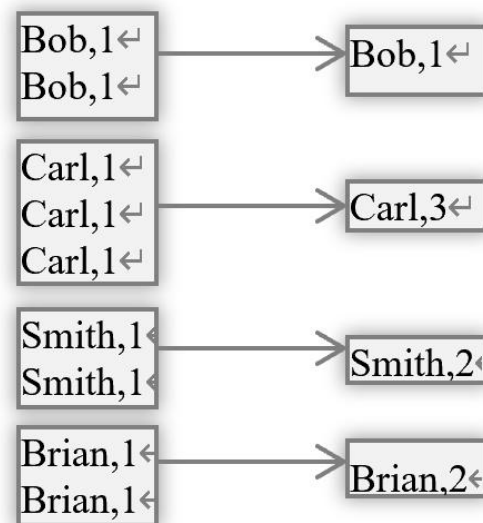
- ◆ MapReduce并行编程模型把计算过程分解为两个主要阶段，即Map 阶段和Reduce 阶段
- ◆ 首先，保存在HDFS里的文件即数据源，已经进行分块。这些数据块交给多个Map 任务去执行，Map任务执行**Map**函数，根据特定规则对数据进行处理，写入本地硬盘
- ◆ Map阶段完成后，进入Reduce 阶段，Reduce任务执行**Reduce** 函数，具有同样Key值的中间结果，从多个Map任务所在的节点，被收集到一起 (**Shuffle**)进行约减处理，输出结果写入本地硬盘(分布式文件系统)
- ◆ 程序的最终结果，可以通过合并所有Reduce 任务的输出得到。需要注意的是，输入数据、中间结果、以及最终结果，都是以<Key, Value>的格式保存到分布式文件系统中，即HDFS中
 - ◆ **Map** 函数处理<Key, Value>对，产生一系列的中间<Key, Value>对
 - ◆ **Reduce** 函数用来合并所有具有相同Key值的中间键值对，计算最终结果
 - ◆ MapReduce计算模型，可以形式化地表达成Map: $\langle k1, v1 \rangle \rightarrow \text{list}\langle k2, v2 \rangle$, Reduce: $\langle k2, \text{list}\langle v2 \rangle \rangle \rightarrow \text{list}\langle k3, v3 \rangle$

Hadoop入门

- ◆ MapReduce计算模型
 - ◆ WordCount实例
 - ◆ Map函数的功能是，对文件块出现的每个单词，输出<单词,1>的键值对
 - ◆ Reduce函数，则把各个Map函数输出的结果，按照单词进行分组，统计其出现的次数



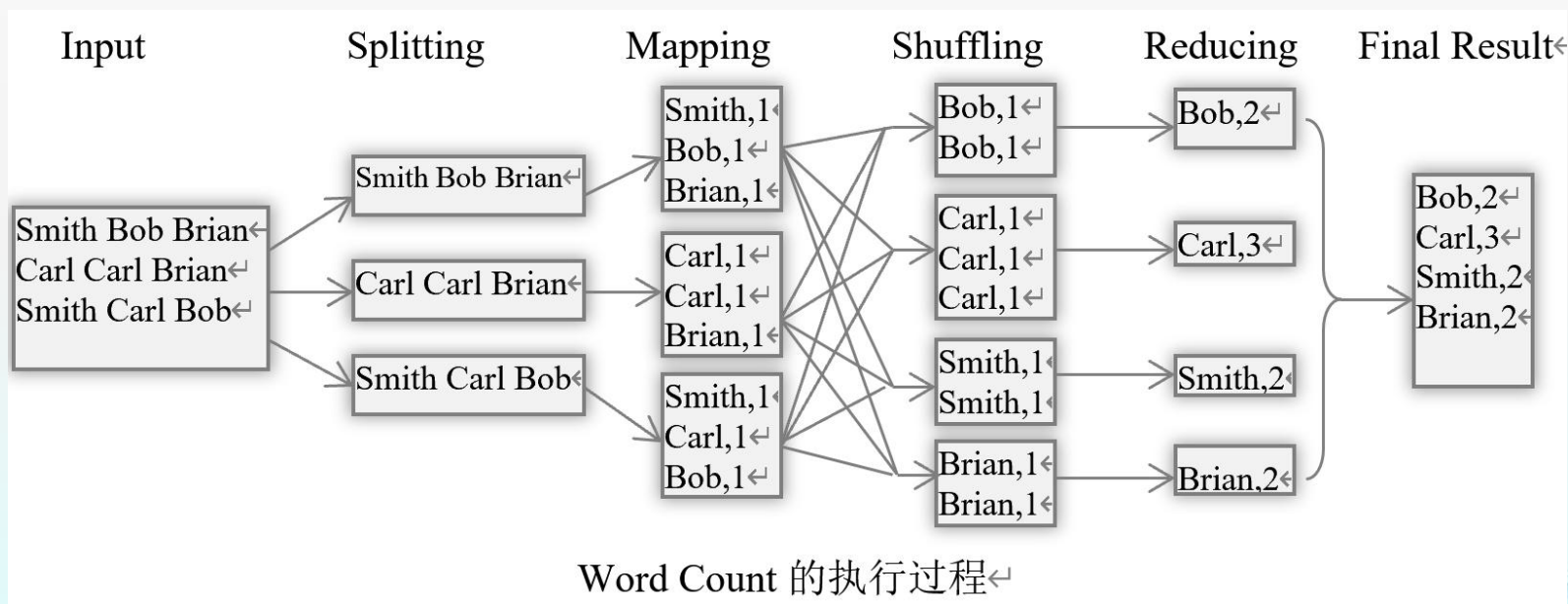
Word Count 的 Map 函数功能



Word Count 的 Reduce 函数功能

Hadoop入门

- ◆ MapReduce计算模型
- ◆ 总流程



pause



Hadoop

◆ MapReduce计算模型的容错性

- ◆ 其思路很简单，出问题的话，重做就是了

- ◆ Map：重新执行

- ◆ Reduce：重新执行

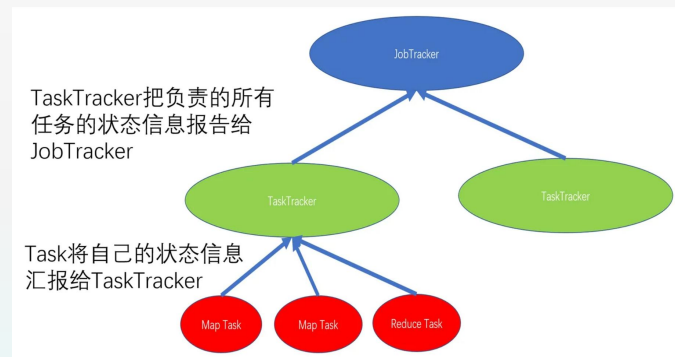
 - ◆ 重新执行的话，需要保存中间数据（Map阶段的中间结果存盘）

 - ◆ Reduce不使用消息机制传递数据，否则Reduce一旦出问题，之前的取得的数据全部浪费了

- ◆ Map和Reduce实现了异步读写，同时其容错也得到了保证

Hadoop

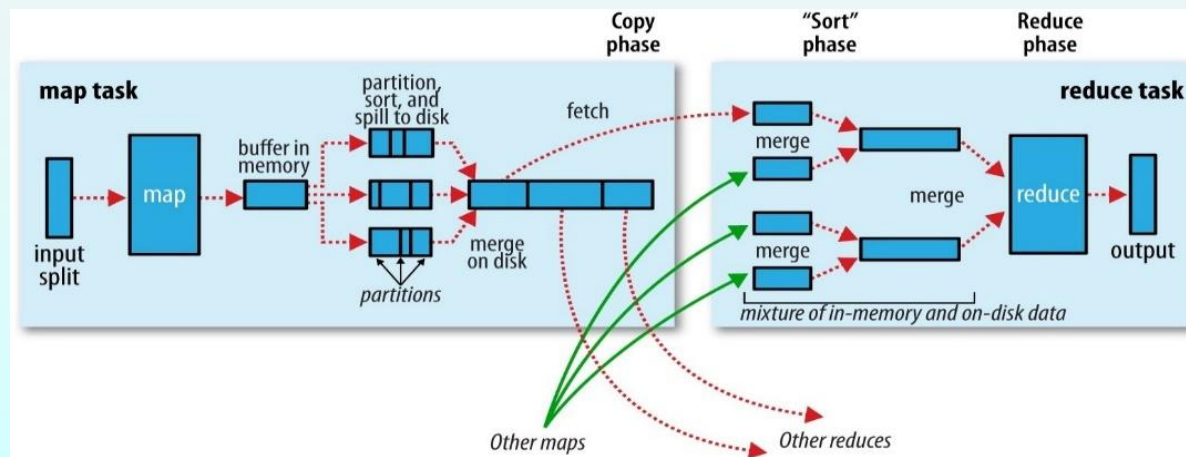
- ◆ MapReduce的某个task执行比较慢straggler
 - ◆ 报告 progress
 - ◆ 在MapReduce中，TaskTracker与JobTracker通过心跳进行通信
 - ◆ TaskTracker在每次心跳中携带关于自己所负责的任务以及所在节点的信息，JobTracker在心跳的回复中告诉TaskTracker接下来该进行的操作
 - ◆ 对一个任务启动备份执行实例
 - ◆ MapReduce早期的判断方法非常简单，就是如果一个任务的执行进度比所属作业的所有任务的平均进度慢20%(即下面代码的SPECULATIVE_GAP)，那么就认为该任务的进度过慢了
 - ◆ 对这两个运行实例，总体的处理原则就是，其中任意一个执行实例成功都代表该任务执行成功
 - ◆ 同时，需要立即杀死另一个正在执行的运行实例



Hadoop

◆ Shuffle

- ◆ 当我们查看Terasort MapReduce应用程序的代码时，没有看到有关的排序功能。但是Terasort程序又能够对大规模的数据进行排序，这是为什么呢？这是由Mapreduce内部运行机制提供保证的。
- ◆ 每个mapper生成的数据首先缓存，缓存达到一定大小后，对数据进行排序，写入磁盘
- ◆ 一系列的磁盘小文件要进行合并，合并的过程，也保持数据的有序性
- ◆ 每个mapper的输出都划分为R份(Segment)，对应R个Reducer task
- ◆ 对于每个reducer，从所有mapper接收应该由它处理的部分(Segment)，这个过程称为Shuffle
- ◆ 然后进行合并，合并的过程保持数据的有序性，即这是一个Merge Sort的过程



pause



Hadoop

- ◆ 用MapReduce实现Join
- ◆ **MapReduce Join** operation is used to combine two large datasets
 - ◆ If one dataset is **smaller** as compared to the other dataset then smaller dataset is **distributed to every data node in the cluster**
 - ◆ Once a join in MapReduce is distributed, either Mapper or Reducer uses the smaller dataset to perform a lookup for matching records from the large dataset and then combine those records to form output

Hadoop

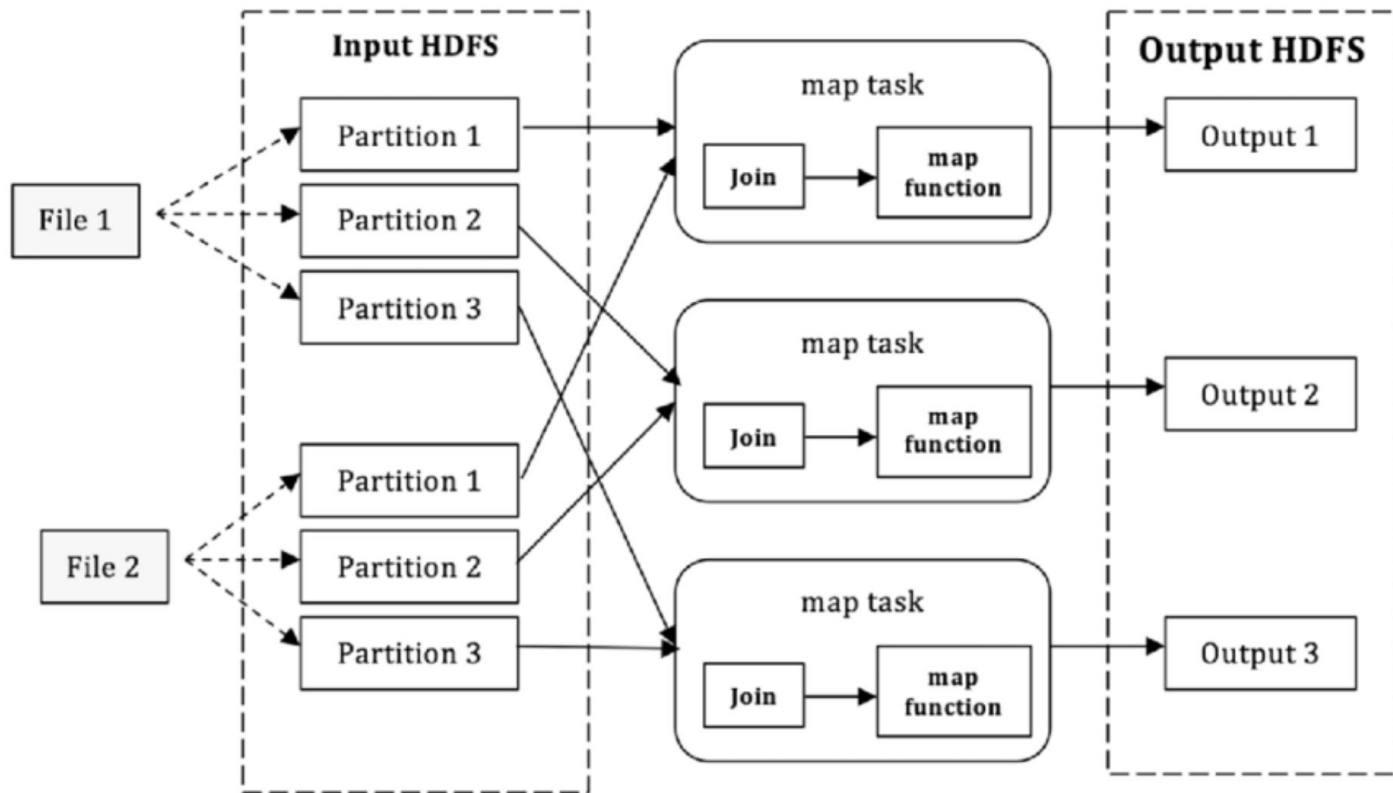
- ◆ 用MapReduce实现Join
- ◆ Depending upon the place where the actual join is performed, joins in Hadoop are classified into
 - ◆ (1) **Map-side join** — When the join is performed by the mapper, it is called as map-side join
 - ◆ In this type, the join is **performed before data is actually consumed by the map function**
 - ◆ It is mandatory that the input to each map is in the form of a partition and is in sorted order; Also, there must be an equal number of partitions and it must be sorted by the join key
 - ◆ Here, **map side processing emits join key and corresponding tuples of both the tables**. As an effect of this processing, all the tuples with same join key fall into the same reducer which then joins the records with same join key

Hadoop

- ◆ 用MapReduce实现Join
- ◆ Map Side Join
 - ◆ You can use Map side join using two different ways based on your datasets, and those depends on below conditions –
 - ◆ Both datasets are must be divided into the **same number of partitions**, and must be already **sorted by the same key**
 - ◆ From the two datasets one must be small(something like master dataset) and able to fit into the memory of each nodes

Hadoop

◆ 用MapReduce实现Join



Map-side join algorithm flowchart

Hadoop

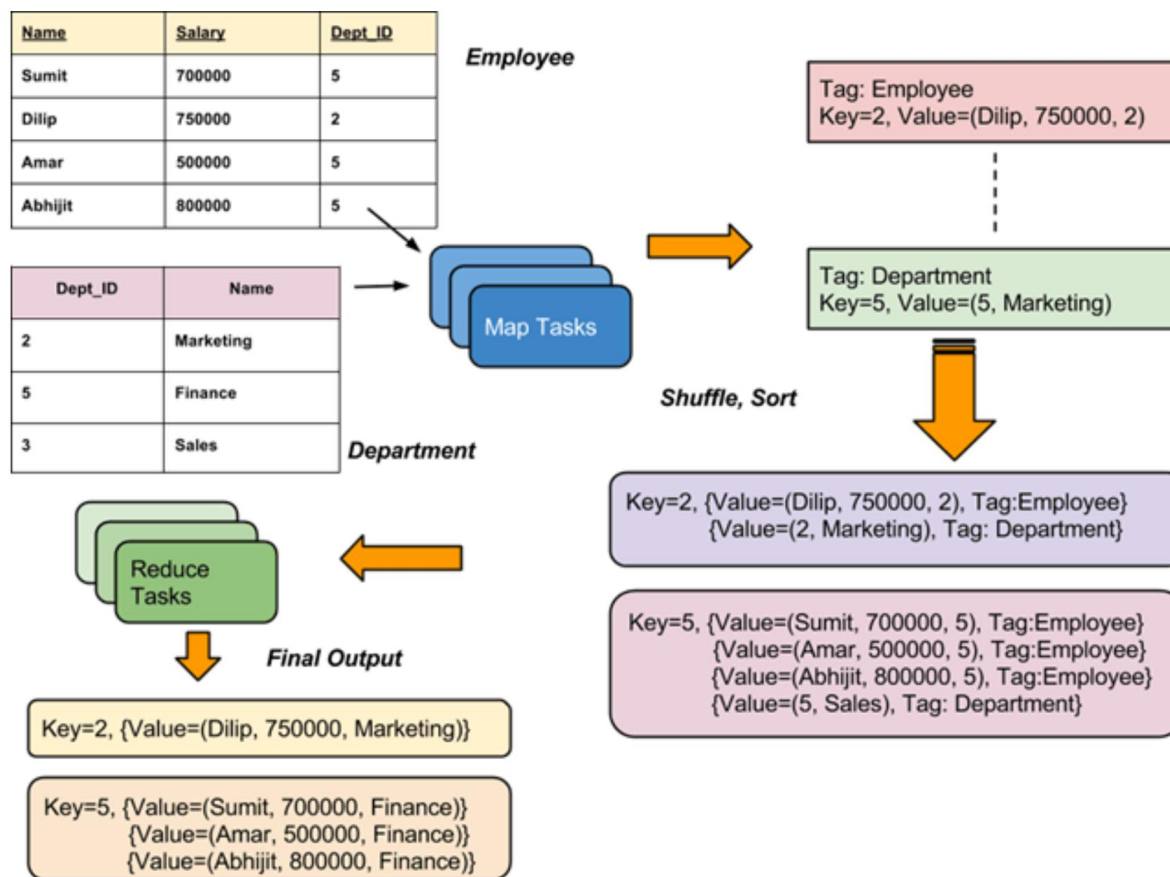
- ◆ 用MapReduce实现Join
- ◆ Depending upon the place where the actual join is performed, joins in Hadoop are classified into
 - ◆ (2) **Reduce-side join** — When the join is performed by the reducer, it is called as reduce-side join. There is no necessity in this join to have a dataset in a structured form (or partitioned).

Hadoop

- 用MapReduce实现Join

<https://www.guru99.com/introduction-to-counters-joins-in-map-reduce.html>

- Reduce Side Join



pause



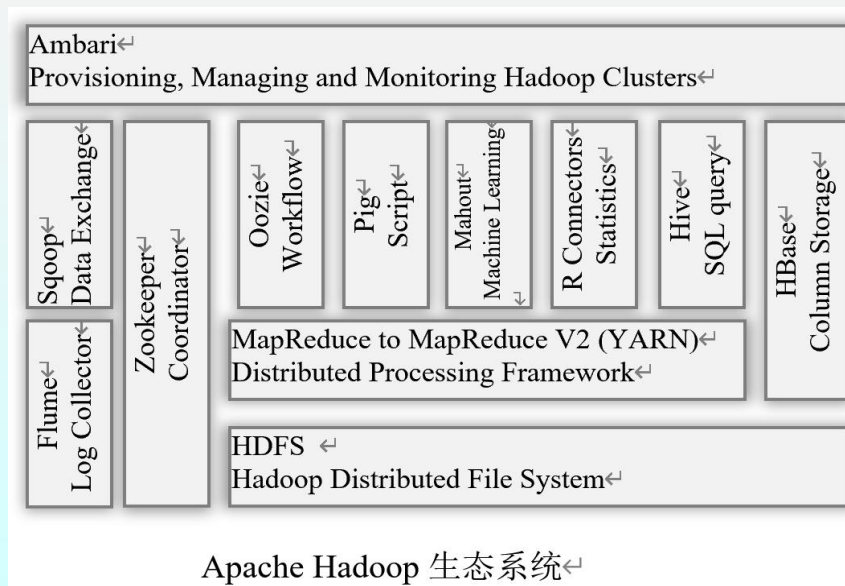
Hadoop入门

- ◆ Hadoop 1.0的应用
 - ◆ MapReduce计算模型，看起来特别的简单。实际上，在这种简单的处理之上，我们可以实现复杂的数据处理任务。
 - ◆ 除了简单的SQL汇总之外，研究人员已经把OLAP、数据挖掘、机器学习、信息检索、多媒体数处理、科学数据处理、图数据处理等复杂的数据处理和分析算法，移植到Hadoop平台上(翻译成MapReduce Job)
 - ◆ Hadoop不仅仅是一个处理非结构化数据的工具，当数据按照一定格式进行适当组织以后，Hadoop平台也可以处理结构化数据
 - ◆ Hadoop平台以及Hadoop上的各种工具构成了一个生态系统，完成各种大数据集的处理任务

Hadoop入门

◆ Hadoop生态系统

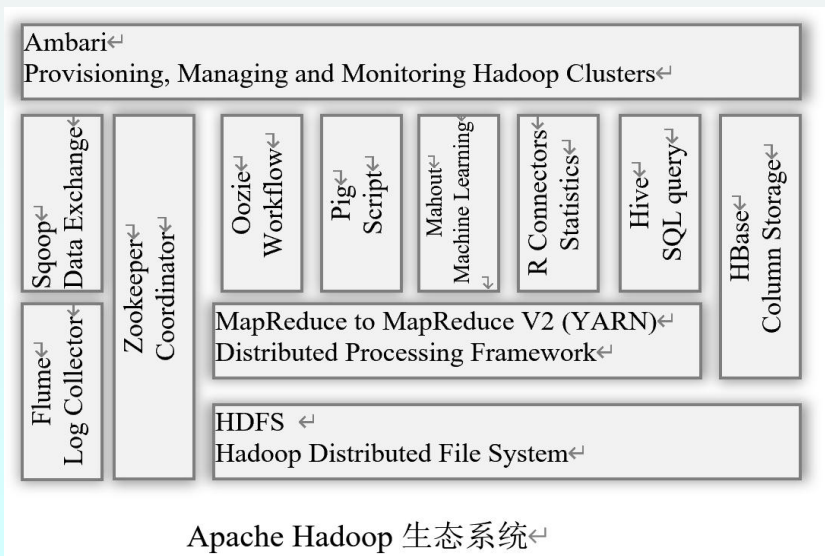
- ◆ **Hive**是Hadoop平台上的数据仓库，用于对数据进行离线分析。它提供了一种类似于SQL的查询语言HQL (Hive Query Language)。Hive将SQL转化为MapReduce作业(Job)在Hadoop上执行。
- ◆ **HBase**是Google Big Table在Hadoop平台上的开源实现。它是一个针对结构化数据处理的、面向列分组(Column Family)的、可伸缩的、高度可靠的、高性能的分布式数据库。一般用于数据服务(Data Serving)应用场合。
- ◆ **Pig**实现了数据查询脚本语言Pig Latin。用Pig Latin脚本语言编写的应用程序，翻译为MapReduce作业，在Hadoop上运行
 - ◆ Pig Latin提供了连接操作，还提供了其它原语操作，方便开发人员编写数据操作算法



Hadoop入门

◆ Hadoop生态系统

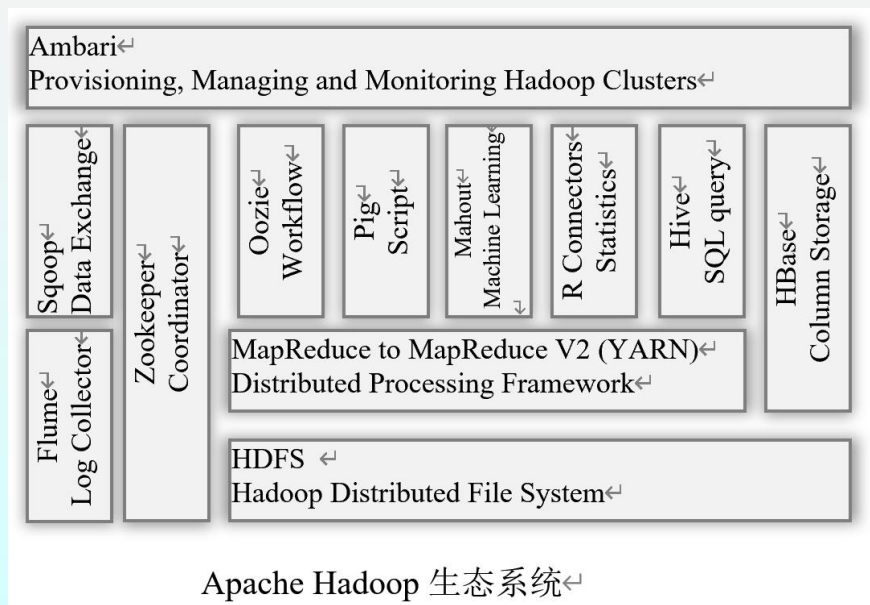
- ◆ **Flume**是一个可扩展的、高度可靠的、高可用的分布式海量日志收集系统，一般用于把众多服务器上的大量日志，聚合到某一个数据中心。Flume提供对日志数据进行简单处理的能力，比如过滤、格式转换等。同时，Flume可以将日志写往各种目标(本地文件、分布式文件系统)。
- ◆ **Sqoop**是SQL to Hadoop的缩写，主要用于在关系数据库或者其它结构化数据源和Hadoop之间交换数据
 - ◆ 比如，Sqoop可以把MySQL等数据库数据导入到Hadoop里，包括HDFS、HBase以及Hive。反过来，它也可以将Hadoop的数据导出到MySQL数据库中
 - ◆ 数据的导入/导出，通过MapReduce作业(应用程序)实现，充分利用了MapReduce的并行化处理能力和容错性能



Hadoop入门

◆ Hadoop生态系统

- ◆ **Mahout**是Hadoop平台上的机器学习软件包，它的主要目标是实现高度可扩展的机器学习算法，以便帮助开发人员利用大数据进行机器学习模型训练。Mahout现在已经包含聚类、分类、推荐引擎(协同过滤)、频繁集挖掘等经典数据挖掘和机器学习算法。
- ◆ **Oozie**是一个工作流调度器(Scheduler)。Oozie协调运行的作业，属于一次性非循环的作业，比如MapReduce作业、Pig脚本、Hive查询、Sqoop数据导入/导出作业等。Oozie基于时间、和数据可用性进行作业调度，根据作业间的依赖关系，协调作业的运行

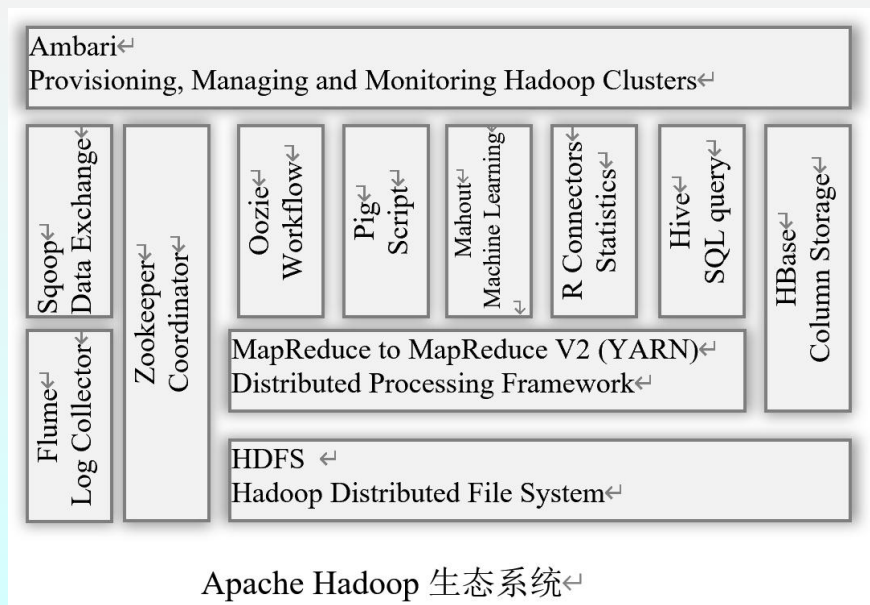


Hadoop入门

- ◆ Hadoop生态系统

- ◆ **Zookeeper**是模仿Google公司的Chubby系统的开源实现，Chubby是一个分布式的锁(Lock)服务

- ◆ 大部分分布式应用都需要这样一些公共服务，包括树状结构的统一命名服务、状态同步服务(通过分布式共享锁)、配置数据的集中管理、集群管理(如集群中节点的状态管理及状态变更通知，节点数据变更的消息通知)等。这些服务，难以实现，也难以调试。借助于Zookeeper，人们就无需为每个分布式应用实现这些功能，加快分布式应用的开发和部署



pause



Hadoop入门

- ◆ Hadoop2.0
 - ◆ 为什么需要集群计算
 - ◆ 第一个因素，是我们是否能够很方便地对系统进行扩展(Cost to Scale)
 - ◆ 由于Hadoop能够运行在由普通服务器构成的超大规模集群上，所以SQL on Hadoop系统，比起传统的MPP数据库比如TeraData、Vertica、Netezza等来讲，具有更强的扩展能力。传统的MPP数据库系统，需要运行在高端服务器上，价格高昂，而且很难扩展到上千个节点。
 - ◆ 第二个因素，是处理大数据时的I/O瓶颈
 - ◆ 当数据的规模足够大，只有一部分数据可以装载到内存中，剩下的数据必须保存在磁盘里。并且在处理过程中，从磁盘上不断装载到内存中，进行后续处理。
 - ◆ 通过把I/O分散到大规模集群的各个节点上，我们可以把数据装载的速度大大提高，进而加快后续的处理。大规模集群把各个节点的I/O带宽，聚集起来，获得比高端服务器大得多的I/O带宽，可以达到几十倍甚至上百倍，无疑是一个廉价有效的大数据处理解决方案

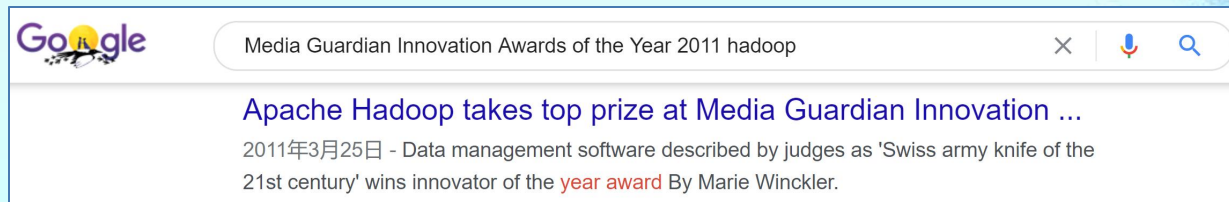
Hadoop入门

◆ Hadoop2.0

◆ Hadoop 1.0的辉煌

◆ 在2008年，Yahoo公司使用一个拥有910个节点的Hadoop集群，在209秒内完成了1TB数据的排序，打破了Terabyte Sort 评测基准的记录(之前的记录是297秒)。这件事件的重要意义在于，这是用Java编写的开源程序，首次赢得Terabyte Sort 评测基准。

◆ 在2011年3月，Media Guardian媒体集团，把年度创新奖(Innovation Awards of the Year)颁发给了Hadoop项目。评审委员会认为，Hadoop项目是“21世纪的瑞士军刀(Swiss Army Knife of the 21st Century)”。Hadoop平台已经成为大数据处理的标准工具，它的重要作用被越来越多的人认识到



Hadoop入门

◆ Hadoop2.0

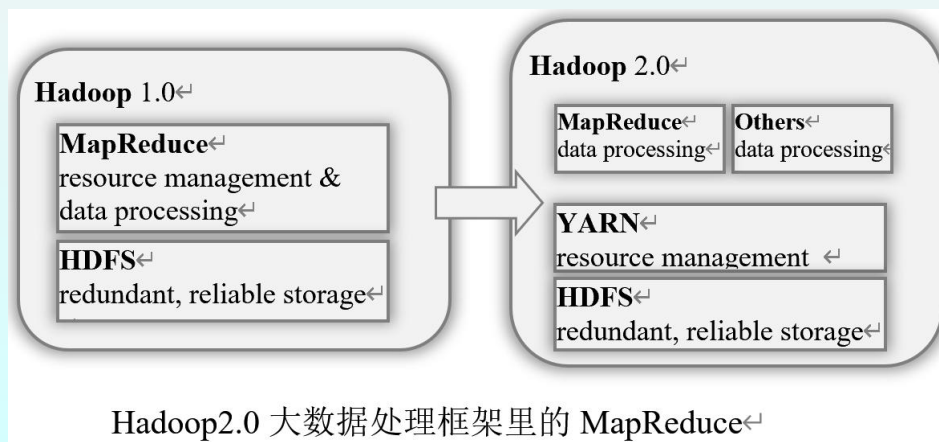
- ◆ Hadoop 1.0的局限，主要的局限包括
 - ◆ (1) 它仅仅支持一种计算模型，即MapReduce。MapReduce计算模型的表达能力有限。复杂的数据处理任务，比如机器学习算法和SQL连接查询等，很难表达为一个MapReduce Job，而是需要翻译成一系列的MapReduce Job，这些Job一个接一个地执行
 - ◆ (2) 由于MapReduce作业在Map阶段和Reduce阶段执行过程中，**需要把中间结果存盘**。而且在MapReduce作业间，也需要通过磁盘实现作业之间的数据交换。通过磁盘进行数据交换，效率低下，影响了查询的执行效率。在这个计算模型上，很难再继续减小查询的响应时间
 - ◆ (3) Hadoop 1.0的**任务调度方法，远未达到优化资源利用率的效果**。在Hadoop1.0版中，对任务的调度方法，即如何给TaskTracker分配任务的过程，比较简单
 - ◆ 每个TaskTracker拥有一定数量的任务槽(Slot)，每个活动的Map或者Reduce任务占用其中一个任务槽。JobTracker把工作分配给最靠近数据的TaskTracker，这个TaskTracker正好有可用的任务槽
 - ◆ 在这个调度方法下，并未考虑将要被分配任务的机器，其当前的系统负载是否过高。如果某个TaskTracker执行非常慢，它将会影响整个MapReduce作业(Job)的执行。整个作业等待最慢的任务完成，才能结束。当然，可以通过猜测执行(Speculative Execution)模式，在多个Slave节点上启动同一个任务(Task)，只要有其中一个任务完成即可

Hadoop入门

- ◆ Hadoop2.0
 - ◆ 从Hadoop 1.0到Hadoop2.0
 - ◆ Hadoop最初是为大数据的**批处理设计的**，它的关注点，在于以尽量高的吞吐量处理这些数据。但是，人们希望Hadoop还能够支持**交互式查询、数据的迭代式处理、以及流数据处理、图数据处理等**
 - ◆ 其中，迭代式处理是机器学习算法所必须的，即机器学习算法一般需要对数据进行多遍扫描和处理
 - ◆ 在这种形势下，Hadoop2.0应运而生。由于Hadoop 2.0的主要改变，是在整个软件架构里划分出了资源管理框架**YARN(Yet Another Resource Negotiator)**
 - ◆ YARN是Hadoop2.0的主要组成部分
 - ◆ 所以，我们有时把YARN和Hadoop2.0互换使用

Hadoop入门

- ◆ Hadoop2.0
 - ◆ 从Hadoop 1.0到Hadoop2.0
 - ◆ Hadoop YARN是Hadoop 2.0的最重要的组成部分。**YARN把资源管理(Resource Management)和作业调度/监控(Job Scheduling/Monitoring)模块分开**。在Hadoop1.0版中，这两个功能都由JobTracker来负责
 - ◆ 在Hadoop1.0中，系统仅仅能够支持一种计算模型即MapReduce。在Hadoop2.0中，**系统可以支持更多的计算模型**，包括流数据处理、图数据处理、批处理、交互式处理等
 - ◆ 在Hadoop2.0中，应用程序可以是传统的MapReduce作业(Job)，或者由一系列任务构成的一个有向无环图(Directed Acyclic Graph, DAG)表达的作业，能够表达更加复杂的数据处理流程



pause



Hadoop入门

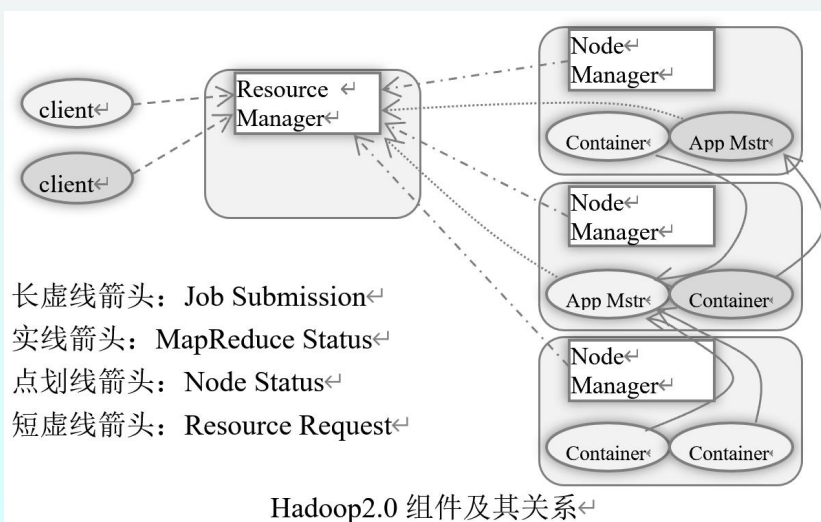
◆ Hadoop2.0

- ◆ Hadoop 2.0的主要组件及其关系

- ◆ 在新的架构里，包含**ResourceManager**和**NodeManager**两个重要的组件。ResourceManager运行在Master节点上，NodeManager运行在Slave节点上，一起负责分布式应用程序的调度和运行

- ◆ **ResourceManager**是为应用程序分配资源的最高权威。ResourceManager包含两个组件，包括Scheduler和ApplicationManager

- ◆ **Scheduler**负责为应用程序分配资源，它根据应用程序的资源需求、以及一些限制条件，包括各个用户的限额等，完成资源的分配和调度。Scheduler使用资源容器(Container)的概念，把CPU、内存、磁盘、网络带宽等资源整合起来



Hadoop入门

◆ Hadoop2.0

- ◆ Hadoop 2.0的主要组件及其关系

- ◆ 在新的架构里，包含ResourceManager和NodeManager两个重要的组件。ResourceManager运行在Master节点上，NodeManager运行在Slave节点上，一起负责分布式应用程序的调度和运行

- ◆ ResourceManager是为应用程序分配资源的最高权威。ResourceManager包含两个组件，包括Scheduler和ApplicationManager

- ◆ ApplicationManager 接受客户端应用程序提交的作业，向Scheduler为该应用程序申请第一个容器，运行针对这个应用程序的ApplicationMaster，用于执行提交的作业(应用程序)，并且在发生失败的情况下，重新启动这个应用程序的ApplicationMaster。

- ◆ ApplicationMaster从Scheduler为应用程序申请资源，和NodeManager一道，在分布式环境下执行应用程序，并追踪其状态、监控作业的进展情况。执行应用程序时，ApplicationMaster 监视容器直到其完成。当应用程序完成时，ApplicationMaster 从ResourceManager 注销其容器，执行周期就完成了。

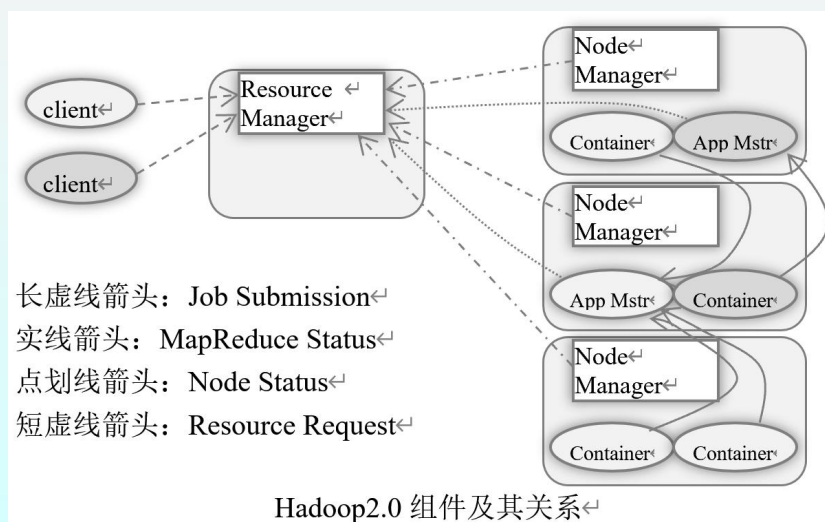
Hadoop入门

◆ Hadoop2.0

◆ Hadoop 2.0的主要组件及其关系

◆ 在新的架构里，包含**ResourceManager**和**NodeManager**两个重要的组件。**ResourceManager**运行在Master节点上，**NodeManager**运行在Slave节点上，一起负责分布式应用程序的调度和运行

◆ **NodeManager**运行在Slave节点上，它为应用程序启动容器，监控其资源使用情况(包括CPU、内存、磁盘、网络带宽的使用情况)，并且把这些信息报告给**ResourceManager**



Hadoop入门

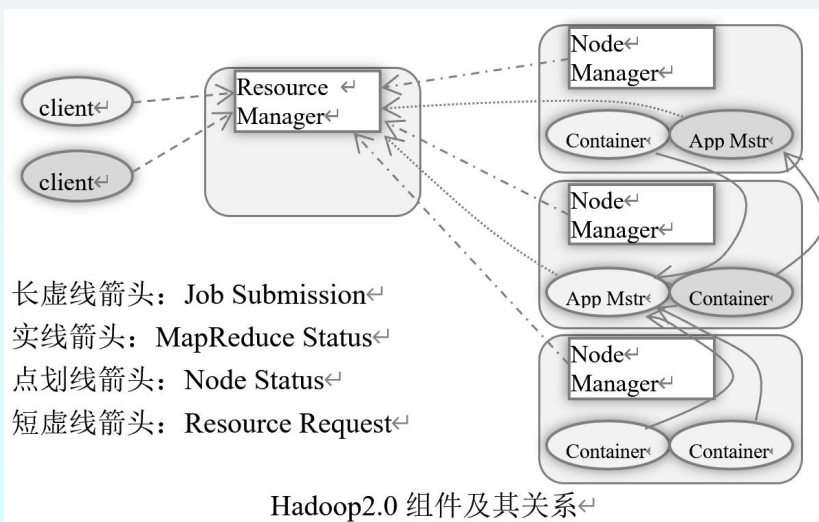
◆ Hadoop2.0

- ◆ Hadoop 2.0的主要组件及其关系

- ◆ 在新的架构里，包含**ResourceManager**和**NodeManager**两个重要的组件。ResourceManager运行在Master节点上，NodeManager运行在Slave节点上，一起负责分布式应用程序的调度和运行

- ◆ **ResourceManager**是为应用程序分配资源的最高权威。ResourceManager包含两个组件，包括Scheduler和ApplicationManager

- ◆ **ApplicationManager** 接受客户端应用程序提交的作业，向Scheduler为该应用程序申请第一个容器，运行针对这个应用程序的ApplicationMaster，用于执行提交的作业(应用程序)，并且在发生失败的情况下，重新启动这个应用程序的ApplicationMaster。



pause



Hadoop入门

◆ Hadoop2.0

- ◆ Hadoop 2.0的主要优势

- ◆ **(1) 扩展性**: ResourceManager的主要功能, 是资源的调度工作。所以它能够轻松地管理更大规模的集群系统, 适应了数据量增长对数据中心的扩展性提出的挑战。

- ◆ **(2) 更高的集群使用效率**: ResourceManager是一个单纯的资源管理器, 它根据资源预留要求、公平性、服务水平协议(Service Level Agreement, SLA)等标准, 优化整个集群的资源, 使之得到很好的利用。

- ◆ **(3) 兼容Hadoop1.0**: 在Hadoop1.0平台上开发的 MapReduce应用程序, 无需修改, 直接在YARN上运行。

- ◆ **(4) 支持更多的负载类型**: 当数据存储到HDFS以后, 用户希望能够对数据以不同的方式进行处理。除了MapReduce应用程序(主要对数据进行批处理), YARN支持更多的编程模型, 包括图数据的处理、迭代式计算模型、实时流数据处理、交互式查询等。一般来讲, 机器学习算法需要在数据集上, 经过多次迭代, 才能获得最终的计算结果。

- ◆ **(5) 灵活性**: MapReduce等计算模型可以独立于资源管理层, 单独演化和改进。使得系统各个部件的演进和配合, 更加具有灵活性

Hadoop入门

- ◆ Hadoop是否已经过时?
 - ◆ Spark 速度更快, Flink批流融合

现在谈大数据, Hadoop已经过时?



小马哥是没有感情的

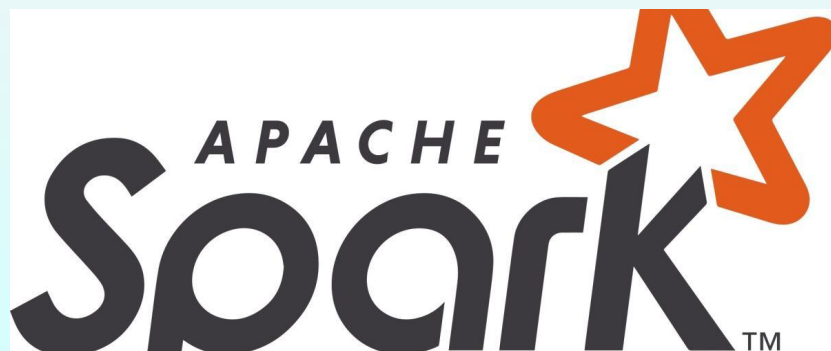
关注



0.072 2019.10.27 11:00:32 字数 3,883 阅读 815

导读: 原先支持Hadoop的四大商业机构纷纷宣布支持Spark, 包含知名Hadoop解决方案供应商Cloudera和知名的Hadoop供应商MapR。

现在再写这篇文章感觉有些不合时宜, 目前, 貌似很少人再讨论大数据, 也很少人再讨论hadoop。整理这篇文章, 是为了探寻新的技术方向。



Copy right π

Hadoop入门

- ◆ Hadoop是否已经过时?
 - ◆ Zookeeper, flume, Sqoop

